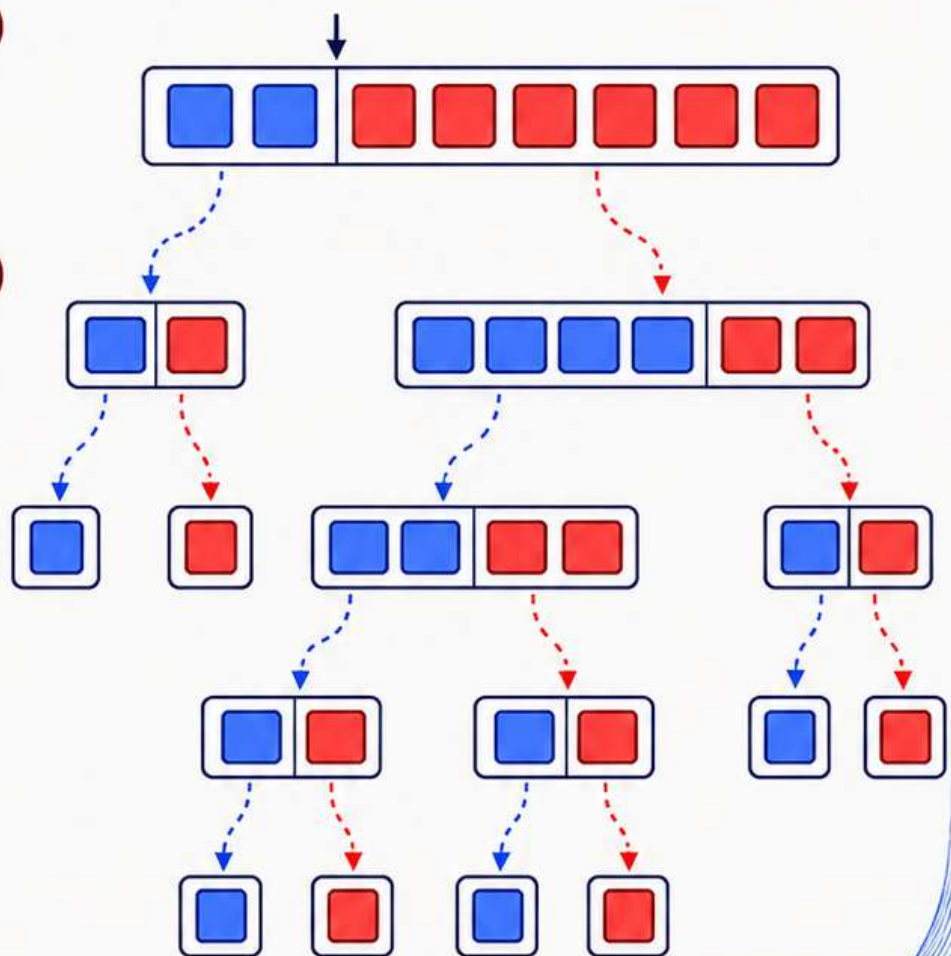
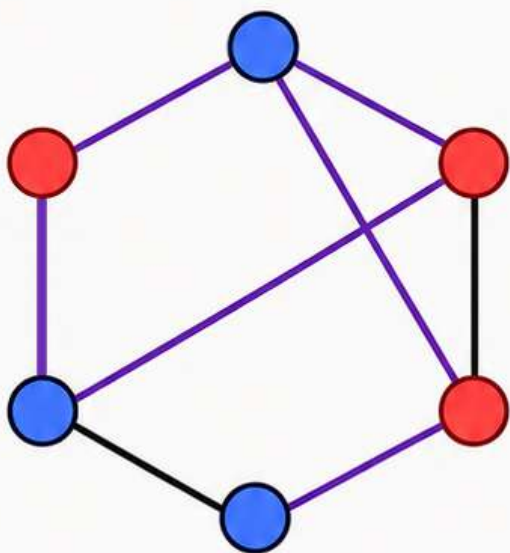


Randomised Algorithms

Sahil M Deo



Contents

1	Introduction	6
1.1	What is a Randomised Algorithm?	6
1.2	Implementation of Randomness in C++	6
1.3	Problems	9
2	Algorithmic Performance	10
2.1	Operations	10
2.2	Worst Case Analysis	10
2.3	Average Case Analysis	11
2.4	Time Complexity Analysis	11
2.5	Problems	13
3	Complexity Classes	14
3.1	Decision Problems	14
3.2	P	14
3.3	NP	15
3.4	Polynomial-Time Reductions	15
3.5	NP-Complete	16
3.6	NP-Hard	16
3.7	RP	17
3.8	ZPP	17
3.9	BPP	18
3.10	Summary	18
3.11	Problems	19
4	The Classics	20
4.1	QuickSort	20
4.1.1	The Algorithm	20
4.1.2	Worst Case Runtime	20
4.1.3	Expected Runtime	21
4.2	QuickSelect	22
4.2.1	The Algorithm	22
4.2.2	Expected Runtime	23
4.2.3	Expected Recursion Depth	23
4.3	Problems	25
5	Hashing	26
5.1	What even is Hashing?	26
5.2	Polynomial Multiplication	26
5.3	Matrix Multiplication	27
5.3.1	Freivalds' Algorithm	27

5.4	A Matrix Problem	28
5.4.1	Set of Sets	29
5.4.2	Hashing	30
5.5	A Sliding Window Problem	31
5.6	Problems	33
6	Data Structures	34
6.1	Skip Lists	34
6.1.1	The Structure	36
6.1.2	Expected Height	36
6.1.3	Search	37
6.1.4	Insertion	39
6.2	Treaps	40
6.2.1	Binary Trees	40
6.2.2	Tree Traversal	41
6.2.3	Binary Search Trees	41
6.2.4	Balancing	45
6.2.5	Min-Heaps	46
6.2.6	Rotations	49
6.2.7	Treaps	50
6.3	Hash Tables	54
6.3.1	The Structure	54
6.3.2	Insertion	54
6.3.3	Search	55
6.3.4	Deletion	56
6.4	Bloom Filters	56
6.4.1	The Structure	56
6.4.2	Insertion	56
6.4.3	Search	57
6.4.4	Utility of Bloom Filters	57
6.5	Cuckoo Hashing	57
6.5.1	The Structure	57
6.5.2	Insertion	58
6.5.3	Search	58
6.6	Problems	59
7	Number-Theoretic Algorithms	61
7.1	Introduction	61
7.2	Fermat Test	61
7.2.1	Fermat's Little Theorem	62
7.2.2	The Algorithm	62
7.2.3	Error Probability	62
7.3	Miller–Rabin Test	63
7.3.1	A Key Property	63
7.3.2	The Algorithm	64
7.3.3	Error Probability	64
7.3.4	Performance	65
7.4	Pollard's Rho Algorithm	65
7.4.1	Floyd's Cycle-Detection Algorithm	66
7.4.2	The Algorithm	68
7.4.3	Performance	69
7.5	Problems	72

8	Probabilistic Method	73
8.1	Introduction	73
8.2	Array Construction	73
8.2.1	Random Experiment	73
8.2.2	Success Probability	74
8.2.3	The Algorithm	75
8.2.4	Expected Time Complexity	76
8.3	Dominating Set	76
8.3.1	Random Experiment	77
8.3.2	Success Probability	77
8.3.3	The Algorithm	78
8.3.4	Expected Time Complexity	78
8.4	Bipartite Subgraph	79
8.4.1	Random Experiment	79
8.4.2	The Algorithm	80
8.4.3	Success Probability	80
8.4.4	Expected Time Complexity	82
8.4.5	Conjecture	82
8.4.6	Derandomisation	82
9	Online Algorithms	84
9.1	Introduction	84
9.2	Dating	84
9.3	Load Balancing	87
9.4	Distinct Elements	89
9.4.1	Flajolet–Martin Method	89
9.4.2	HyperLogLog Method	91
10	Heuristic Algorithms	92
10.1	Introduction	92
10.2	Dominating Set Revisited	92
10.3	Independent Set	94
10.3.1	Random Permutation	94
10.3.2	Local Decision	95
10.4	Bipartite Subgraph Revisited	97
10.4.1	Self-Correction	97
11	Game Theoretic Algorithms	98
11.1	Definitions	98
11.2	Nash Equilibria	98
11.2.1	Kingdom’s Dilemma	99
11.2.2	Penalty Kick	99
11.3	Mixed Strategies	100
11.4	Mixed Strategy Nash Equilibria	100
11.5	Indifference Theorem	101
11.6	The Minimax Principle	101
11.7	Problems	102

12 Zero-Knowledge Proofs	103
12.1 Definitions	103
12.2 Card Color	104
12.2.1 The Scheme	104
12.2.2 Completeness and Soundness	104
12.3 Graph Isomorphism	104
12.3.1 The Scheme	105
12.3.2 Soundness and Completeness	105
12.3.3 Game Theory Perspective	106
13 Derandomization	107
13.1 Introduction	107
13.2 Independence	107
13.2.1 Mutual Independence	107
13.2.2 k -Wise Independence	107
13.3 Generating Independent Bits	107
13.4 Bipartite Subgraph Re-revisited	108
13.5 Pairwise Independence	108
13.6 MAX-CUT with Only $\Theta(\log n)$ Random Bits	108
13.7 k -Wise Independence	109
13.8 Polynomial Construction of k -Wise Independent Variables	109
13.9 Why Use k -Wise Independence?	109
13.10 The Method of Conditional Expectations	110
13.11 Pseudorandom Generators	110
13.12 Expander Walks	110
13.13 The Complexity-Theoretic View	110
Appendices	107
A Probability and Expectation	108
A.1 Linearity of Expectation	108
A.2 Tail Sum Formula	108
A.3 Union Bound	108
A.4 Independence of Expectation	108
A.5 Variance	108
A.6 Indicator Variables	109
A.7 Law of Total Expectation	109
A.8 Wald's Equation	109
B Inequalities	111
B.1 Exponential Inequality	111
B.2 Jensen's Inequality	111
B.3 Markov's Inequality	112
B.4 Bernoulli's Inequality	113
C Convergence and Limit Theorems	114
C.1 Identical and Independently Distributed Random Variables	114
C.2 Almost Surely	114
C.3 Almost Never	114
C.4 $\lim_{n \rightarrow \infty} X_n = Y$	115
C.5 Convergence in Probability	115
C.6 The Weak Law of Large Numbers	115

C.7	Almost Sure Convergence	116
C.8	A_n infinitely often	116
C.9	The Borel–Cantelli Lemmas	116
C.10	The Strong Law of Large Numbers	117
C.11	Convergence in Distribution	117
C.12	The Central Limit Theorem	118
D	Discrete Mathematics	119
D.1	Pigeonhole Principle	119
D.2	Relations	119
D.2.1	Partial Orders	119
D.2.2	Equivalence Relations	120
D.3	Graph Theory	120
D.3.1	Degree	120
D.3.2	Bipartite Graph	120
D.3.3	Subgraph	120
D.3.4	Clique	120
E	Linear Algebra	121
E.1	Vector Spaces	121
E.2	Subspaces	121
E.3	Dimension of a Subspace	121
E.4	The Nullspace	122
E.5	Nullity and Rank	122
E.6	The Rank–Nullity Theorem	122

Chapter 13

Derandomization

13.1 Introduction

Randomized algorithms often achieve remarkable simplicity and efficiency. However, they raise a natural question how much randomness is really necessary? Recall that we stated that for our purposes, pseudo-randomness is true randomness. In this chapter, we will finally differentiate between the two! Since true randomness is often expensive or unavailable, it is natural to ask whether one can reduce, or even eliminate, the amount of randomness required. The study of such questions forms the subject of *derandomization*.

13.2 Independence

13.2.1 Mutual Independence

A set of random variables $S = \{X_1, X_2, \dots, X_n\}$ are said to be *mutually independent* if for every subset $s \subseteq S$, we can write the probability of a particular assignment of values as the product of the individual probabilities.

$$\begin{aligned} \{X_{i_1}, X_{i_2}, \dots, X_{i_t}\} &\subseteq \{X_1, X_2, \dots, X_n\} \\ \implies \Pr[X_{i_1} = a_1, \dots, X_{i_t} = a_t] &= \prod_{j=1}^t \Pr[X_{i_j} = a_j] \end{aligned}$$

This definition is the strongest possible definition of independence and is the default meaning of the term “independent”.

13.2.2 k-Wise Independence

A set of random variables $X_1, X_2, \dots, X_n$ are said to be *k-wise independent* if every subset of at most k variables is mutually independent.

By definition it follows that $k+1$ -wise independence implies k -wise independence. Of course, n -wise independence is the same as the whole set being independent and is thus the strongest version of k -wise independence. The case $k = 2$ is the weakest version of k -wise independence and is called *pairwise independence*.

13.3 Generating Independent Bits

Let $k = \lceil \log_2 n \rceil$.

Choose uniformly random values

$$a, b \in \mathbb{F}_{2^k}.$$

For every vertex v_i , identify i with an element of $\mathbb{F}_{2^k}$ and define

$$X_i = a \cdot i + b.$$

The least significant bit of X_i determines the side of the cut.

Only $2k = \Theta(\log n)$ random bits are required. The variables generated above are pairwise independent.

Fix distinct vertices $i \neq j$.

The equations

$$ai + b = x$$

and

$$aj + b = y$$

have a unique solution (a, b) over $\mathbb{F}_{2^k}$.

Hence every pair (x, y) occurs with equal probability.

Therefore the corresponding output bits are pairwise independent.

13.4 Bipartite Subgraph Re-visited

Recall the probabilistic method of simply generating a random 2-coloring of the vertices and then checking the number of bichromatic edges. For every run of that method, we toss a coin for each vertex. This corresponds to $n = |V|$ random bits. In our proof that the expected $\mathbb{E}[X]$ is at least $m/2$, did we really need all n bits to be completely independent?

13.5 Pairwise Independence

Observe carefully that the analysis of MAX-CUT only used the fact that, for every edge (u, v) ,

$$\Pr[u \text{ and } v \text{ are placed on opposite sides}] = \frac{1}{2}.$$

To establish this, it suffices that the random choices made for any pair of vertices are independent.

Full independence among all n vertices is unnecessary.

Random variables $X_1, \dots, X_n$ are said to be *pairwise independent* if for every distinct i, j ,

$$\Pr[X_i = a, X_j = b] = \Pr[X_i = a] \Pr[X_j = b]$$

for all possible values a, b .

13.6 MAX-CUT with Only $\Theta(\log n)$ Random Bits

Replacing the fully independent coin flips by pairwise independent ones preserves the original analysis.

For every edge e ,

$$\Pr[e \text{ is cut}] = \frac{1}{2}.$$

Hence

$$\mathbb{E}[\text{Cut Size}] = \frac{m}{2}$$

still holds.

Thus we obtain:

There exists a randomized algorithm for MAX-CUT using only $\Theta(\log n)$ random bits while guaranteeing an expected cut size of at least $m/2$.

13.7 k -Wise Independence

Pairwise independence is merely the first step.

Random variables $X_1, \dots, X_n$ are said to be *k -wise independent* if every subset of at most k variables is jointly independent.

13.8 Polynomial Construction of k -Wise Independent Variables

Let $\mathbb{F}_q$ be a finite field with $q \geq n$.

Choose uniformly random coefficients

$$a_0, a_1, \dots, a_{k-1} \in \mathbb{F}_q.$$

Define

$$f(x) = a_0 + a_1x + \dots + a_{k-1}x^{k-1}.$$

For each vertex i , set

$$X_i = f(i).$$

The random variables $X_1, \dots, X_n$ are k -wise independent.

Fix distinct points

$$i_1, \dots, i_k.$$

For arbitrary values

$$y_1, \dots, y_k,$$

there exists a unique polynomial of degree at most $k - 1$ satisfying

$$f(i_j) = y_j$$

for every j .

Since all coefficient vectors are equally likely, every assignment to the chosen variables occurs with equal probability.

Hence the variables are k -wise independent.

13.9 Why Use k -Wise Independence?

Many probabilistic analyses depend only on small groups of variables.

Examples include:

- variance calculations — pairwise independence suffices,
- Chebyshev bounds — pairwise independence suffices,

- higher moment methods — require higher-order independence,
- Chernoff-type bounds — often require approximately $O(\log n)$ -wise independence.

13.10 The Method of Conditional Expectations

...

13.11 Pseudorandom Generators

...

13.12 Expander Walks

...

13.13 The Complexity-Theoretic View

$$\mathbf{P} \stackrel{?}{=} \mathbf{BPP}$$

remains one of the central open problems in complexity theory.
Most researchers conjecture that

$$\mathbf{P} = \mathbf{BPP}.$$